CS575 Operating Systems
Homework 4
3/25/2025
Joseph Rissman
U09545917

(a) Identify the data involved in the race condition.

The data involved in the race condition is the variable available_resources. Multiple processes can access and modify this data concurrently.

(b) Identify the location(s) in the code where the race condition occurs.

The race condition occurs within the decrease_count() function on lines 5 and 8.

```
 1  /* decrease available_resources by count resources */
 2  /* return 0 if sufficient resources available, */
 3  /* otherwise return -1 */
 4  int decrease_count(int count) {
 5      if (available_resources < count)
 6          return -1;
 7      else {
 8          available_resources -= count;
 9          return 0;
10      }
11  }
```

This is because the function checks the value of available_resources on line 5, but a context switch can occur before the process can decrement the value on line 8. This can lead to a situation where more resources are allocated than are available if the other process also enters the decrease_count() function.

The increase_count() function can also contribute to a race condition in combination with the decrease_count() function. If one process is in the middle of the decrease_count() function and has already checked the value of available_resources, then a context switch occurs to a process that calls the increase_count() function, the value of available_resources can change unexpectedly, resulting in inconsistent data.

(c) Using a semaphore or mutex lock, fix the race condition. It is permissible to modify the decrease_count() function so that the calling process is blocked until sufficient resources are available.

One way to fix the race condition is with a semaphore lock. The semaphore resource_lock is declared and initialized to 1 using sem_init() at the start of the program. Both functions use sem_wait() to attempt to access the semaphore before accessing available_resources. If the semaphore is available, the process acquires it then proceeds. If the semaphore is not available,

the process is blocked until another process releases the semaphore. After modifying available_resources, sem_post() is called to release the semaphore, allowing another process to acquire it.

This fixes the race condition because the semaphore ensures that only one process can access and modify available_resources at any given time. The critical sections of code where available_resources is accessed and modified become atomic, so they execute as a single, indivisible operation. This prevents the interleaved execution that leads to the race condition.

```
#include <semaphore.h>
#define MAX_RESOURCES 5
int available_resources = MAX_RESOURCES;
sem_t resource_lock;
void initialize_resources() {
    sem_init(&resource_lock, 0, 1);
}
int decrease_count(int count) {
    sem_wait(&resource_lock);
    if (available_resources < count) {
        sem_post(&resource_lock);
        return -1;
    } else {
        available_resources -= count;
        sem_post(&resource_lock);
        return 0;
    }
}
int increase_count(int count) {
    sem_wait(&resource_lock);
    available_resources += count;
    sem_post(&resource_lock);
    return 0;
}
```